v1 → v2 → v3

ClaimCheck Daily

Agentic AI Fact-Checking System

Three generations of a live fact-checking pipeline

Bharani Rajendran · Grad5900 Agentic AI

The Challenge: Automated Fact-Checking

The Problem

1. News publishes faster than humans can verify
2. Verdicts need grounding in evidence, not LLM intuition alone
3. Evidence from one claim should be reusable in the next run
4. The system must be accessible as a tool, not just a CLI

Our Approach

Multi-model pipeline

Claude Opus researches · GPT-4o selects & judges

Persistent evidence store

Chunks reused across runs via hybrid RAG

Self-correcting loop

LLM-as-Judge triggers targeted retries

MCP server interface

Any MCP host can call check_claim()

Three Generations at a Glance

v1 Basic Fetch Agent

- Single Claude agent
- RSS feed → fetch_url loop
- Raw research findings output
- No evidence persistence
- No verification step

v2 Multi-Agent + RAG

- GPT-4o Director added
- Persistent evidence store
- Hybrid TF-IDF + BM25 retrieval
- Knowledge graph context
- LLM-as-Judge + retry loop

v3 Full Pipeline + MCP

- MCP server interface
- Review gate for low-confidence
- web_search via Serper API
- Full 11-node LangGraph
- Public tool access

How v1 Works

v1

Basic Fetch Agent

Harvest	Parse RSS/Atom feeds — extract headline + URL as a Claim object
Select	GPT-4o Director reviews candidates and picks the most verifiable
Research	Claude Opus: fetch_url loop reads the source article → raw findings
Verdict	GPT-4o Director synthesizes a verdict label + summary from findings
Publish	HTML and JSON reports written to docs/ and outputs/

v1 — Components & What Was Missing

v1

Components

Director (GPT-4o)
Selects claims from RSS
Synthesizes verdict text

Researcher (Claude)
ReAct fetch_url loop
Returns raw research text

fetch_url Tool
httpx + BeautifulSoup
Strips HTML → plain text

Publisher
Writes HTML + JSON
to docs/ and outputs/

Limitations Discovered

- ✕ Each run starts fresh — no evidence reuse across runs
- ✕ Claude guesses source URLs; no live web search
- ✕ No quality check on verdicts — hallucinations pass through
- ✕ One LLM both researches and judges — no independence

What v2 Added

v2

Multi-Agent + RAG

Evidence Store	Persistent JSON store — chunks survive across runs and feed into retrieval
Hybrid Retrieval	TF-IDF (60%) + BM25 (40%) with claim-local-first merge strategy
Knowledge Graph	networkx graph tracks source domains — adds cross-run context to queries
Verifier	Independent GPT-4o judge scores verdicts on 4 RAGAS dimensions
Retry Loop	Weak verdicts trigger revise_query → re-retrieve → re-verdict cycle

v2 — Components in Detail

v2

Director (GPT-4o)

- Claim selection from RSS candidates
- Verdict synthesis from retrieved evidence

Researcher (Claude)

- fetch_url ReAct loop
- Fetched pages stored for chunking

Evidence Store

- raw_source ×1.15 · summary ×1.0
- source_metadata ×0.75 chunk weights

HybridRetriever

- TF-IDF cosine + BM25 keyword
- Claim-local boost (SAME_CLAIM ×1.25)

Knowledge Graph

- networkx source-domain nodes
- Adds related context to retrieval

Verifier (GPT-4o)

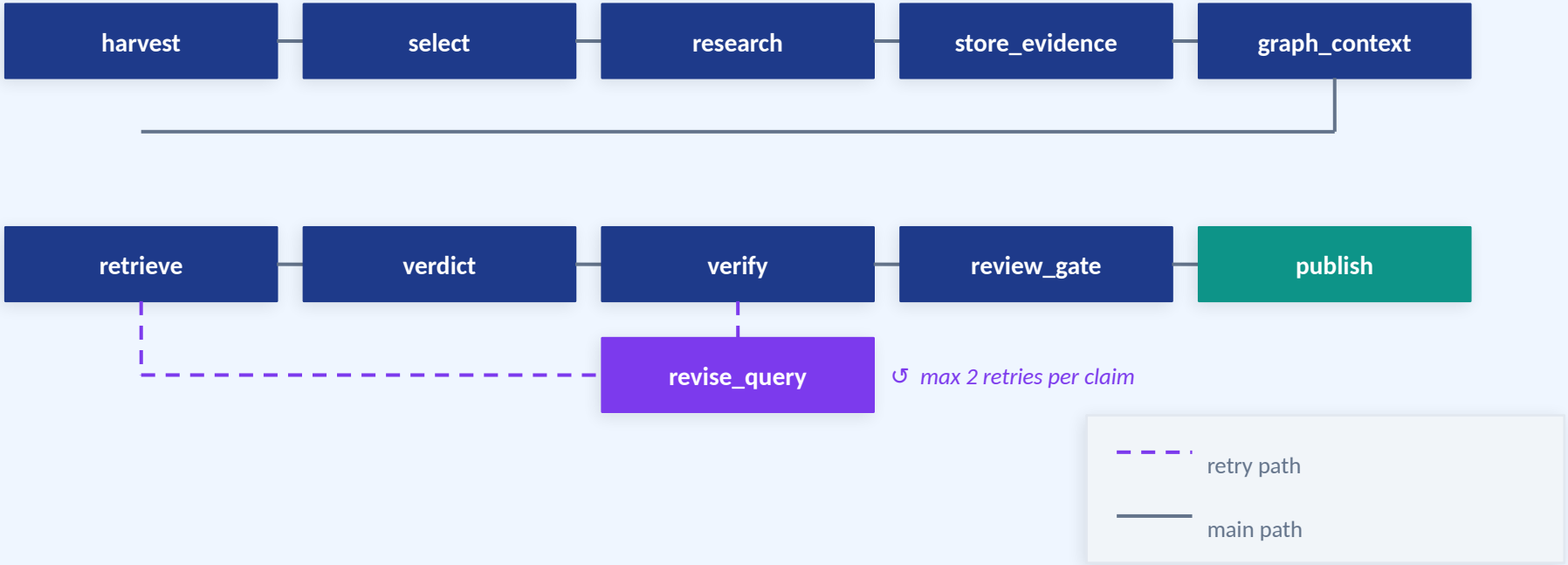
- 4 RAGAS dimensions:
- Groundedness · Citation · Contradiction · Assumption

v2 → v3 — What Still Needed Work

v2 Gaps	v3 Solutions
✕ No external access — check_claim required running the CLI	✓ MCP server — check_claim(), search_evidence(), get_verdict_history()
✕ Low-confidence verdicts published without a safety net	✓ Review gate — MIXED/UNVERIFIABLE claims go to review_queue.json
✕ Researcher guesses URLs — no live web search capability	✓ web_search tool — Serper API finds authoritative URLs for Claude
✕ No single-claim invocation path for interactive use	✓ CLI --claim flag + MCP path for interactive single-claim checks

11-Node LangGraph Pipeline

v3



v3 — 11-Node LangGraph: What Each Node Does

v3

harvest_node

Parses RSS/Atom feeds
Produces candidate Claim objects

select_node

GPT-4o Director picks top claims
Filters by checkability

research_node

Claude: web_search → fetch_url loop
Returns ResearchResult + fetched pages

store_evidence_node

Chunks pages into raw_source, summary, source_metadata types

graph_context_node

Updates networkx domain graph
Adds related-claim context to state

retrieve_node

Hybrid TF-IDF + BM25 retrieval
Claim-local-first merge

verdict_node

GPT-4o synthesizes grounded verdict
Labels: TRUE/FALSE/MIXED/etc.

verify_node

LLM-as-Judge: 4 RAGAS dimensions
Triggers retry if weak

revise_query_node

Refines retrieval query for retry
Targeted re-fetch if needed

review_gate_node

Queues low-confidence claims
Persists review_queue.json

publish_node

Writes HTML to docs/
Writes JSON to outputs/

Research Engine — web_search + fetch_url

v3

web_search (new in v3)

Tool:	Serper API → Google organic results
Input:	search query string
Output:	top-5 results — title, URL, snippet
Purpose:	Claude discovers authoritative URLs instead of guessing from training knowledge

fetch_url

Tool:	httpx + BeautifulSoup
Input:	URL string
Output:	plain text up to 4,000 chars
Purpose:	Read the full article content that web_search found

ReAct Tool-Use Loop (up to 5 rounds for MCP · 8 rounds for CLI)

Claim + URL — web_search — fetch_url — Reason — fetch_url #2 — Findings

Claude can loop, backtrack, or skip steps — extended thinking guides the strategy

Evidence Store & Hybrid Retrieval

Evidence Store

raw_source ×1.15

Full article text — highest weight,
most direct connection to the claim

summary ×1.00

Researcher findings summary —
normalized baseline weight

source_metadata ×0.75

Title + URL + reliability note —
lightest weight, structural signal

Hybrid Retrieval Strategy

TF-IDF 60% cosine similarity · term frequency

BM25 40% keyword scoring · exact-match boost

Merge Strategy

Claim-local hits first · SAME_CLAIM boost ×1.25
Cross-run fallback · Domain cap MAX_HITS_PER_DOMAIN = 2

Retriever Cache (v3 optimization)

LRU cache keyed by chunk fingerprint — avoids O(n) TF-IDF rebuild
on repeated retrieval

Why Persistence Matters

In an era where articles get quietly edited, deleted, or taken down under political pressure...

ChatGPT / stateless AI

- Starts from zero every session
- No record of what an article said yesterday
- If a site deletes or edits a story — the evidence is gone
- Cannot verify what was claimed at a specific point in time

ClaimCheck / persistent store

- Captures source text at research time
- Evidence stored with URL + date provenance
- Article deleted tomorrow? Evidence was already captured
- Day 30 has access to all evidence from days 1-29

The Wayback Machine exists for exactly this reason. ClaimCheck applies the same principle to evidence — capture at the moment of research, store with provenance, retrieve later.

Self-Correcting Verification Loop

LLM-as-Judge — 4 RAGAS Dimensions

Groundedness
Are claims supported by retrieved evidence?

Citation quality
Are specific sources cited correctly?

Contradiction
Does the verdict contradict any evidence?

Assumptions
Does the verdict rely on unverified assumptions?

Retry Flow

verdict — verify — review_gate — publish

PASS → verdict → verify → review_gate → publish

FAIL → verify ✗ → revise_query → re-retrieve → verdict

Max 2 retry iterations per claim

Synthesis Deadline (v3 optimization)

Hard stop injected at round 3 (MCP) or round 6 (CLI).

Forces Claude to write final findings instead of burning remaining rounds and returning "Research incomplete."

TRUE	MOSTLY_TRUE	MIXED	MOSTLY_FALSE	FALSE	UNVERIFIABLE

MCP Server — check_claim, search_evidence, get_verdict_history

v3

check_claim(text)

- Run the full pipeline on a single claim
- Returns: verdict, confidence, summary, key_evidence
- verifier_score, cost_summary, retrieved_evidence
- Writes to docs_manual/ and outputs_manual/

search_evidence(query, k)

- Search the cumulative evidence store
- Hybrid TF-IDF + BM25 across all past runs
- Returns top-k matching chunks with scores
- Useful to explore past evidence before a full check

get_verdict_history(date)

- Retrieve past verdicts from JSON archive
- Pass YYYY-MM-DD or omit for latest run
- Searches both daily and manual archives
- Returns claim, verdict, confidence, verifier_score

MCP Timeout Tuning

Claude Desktop has a ~60 s request timeout. The full pipeline with retries takes 80–120 s.

```
CLAIMCHECK_MAX_TOOL_ROUNDS = 5
```

Research capped at 5 rounds for MCP (vs 8 for CLI) — saves ~30s

```
CLAIMCHECK_MAX_RETRIES = 0
```

Retry / corroborate loop disabled for MCP — saves another ~40s

Both defaults can be overridden in .env. CLI daily runs use 8 rounds + 2 retries (unaffected).

v3 Runtime Optimizations

v3

Retriever Cache

HybridRetriever cached by MD5 chunk fingerprint (count + first/last chunk_id). Eliminates repeated O(n) TF-IDF fit_transform — each unique chunk set is built once and reused for the life of the process.

Synthesis Deadline

Hard stop injected at round MAX_TOOL_ROUNDS - 2. At round 6 (CLI) or round 3 (MCP), Claude is told to stop fetching and write its final report. Prevents the "Research incomplete" fallback when all rounds are exhausted.

MCP Timeout Tuning

mcp_server.py sets CLAIMCHECK_MAX_TOOL_ROUNDS=5 and CLAIMCHECK_MAX_RETRIES=0 before the Researcher singleton is created. First-pass research + verdict completes in ~35–40 s, well under the 60 s MCP client timeout.

Director Claim-Framing Guard

Evaluative meta-claim detector (_looks_like_evaluative_meta_claim) identifies headlines like "X Amplifies Baseless Claim About Y". Extra prompt guidance keeps GPT-4o judging the full headline, not the embedded proposition.

What Was Built

v1 Claude fetches source article · GPT-4o selects & writes verdict · No RAG · No persistence

v2 Persistent evidence store · Hybrid TF-IDF + BM25 retrieval · LLM-as-Judge + retry loop

v3 11-node LangGraph pipeline · web_search + fetch_url tools · MCP server · Review gate

11	2	3	TF-IDF +BM25	4
LangGraph nodes	LLM providers	MCP tools	hybrid retrieval	RAGAS dimensions

github.com/bharanirajendran-blip/ClaimCheck_Daily_v3

"ChatGPT gives you an answer.

ClaimCheck gives you a system."

Persistent memory

Evidence accumulates across runs — not wiped each session

Independent verdict

Claude researches, GPT-4o judges, separate GPT-4o verifies

Self-correcting

RAGAS verifier scores verdicts and triggers retries on failure

Fully automated

Ingests RSS feeds and publishes reports daily — no human needed